Microarchitecture Design

1. Introduction

1.1 Processor Context and Target Application Domain

This document specifies the microarchitecture that Support "SesothoAI-v1", a custom 32-bit RISC-style processor. The design is purpose-built for a highly specific application context: powering **low-cost, AI-enabled mobile phones tailored for the Lesotho market**.

The core mission of this processor is to bridge accessibility gaps by running AI-driven applications that are computationally intensive but must be executed at an affordable, power-constrained device. Our target application domain is **Voice and Language Processing for Local Accessibility**. This includes computationally demanding workloads such as:

1. **Real-time Sesotho Voice Dictation:** Enabling users to interact with their devices, send messages, and use services using their native language, overcoming literacy or digital interface barriers.
2. **Noise-Robust Voice Biometric Security:** Providing a secure and accessible alternative to PINs or passwords for services like mobile money (M-Pesa) and social grant verification.
3. **On-Device Translation/Read-Aloud Assistants:** Supporting communication and information access for citizens and tourists alike.

These applications are currently non-existent or perform poorly on low-end devices, yet they represent a significant opportunity for social and economic inclusion.

1.2 Reference ISA

This microarchitecture is a direct implementation of our custom **"SesothoAI-v1" Instruction Set Architecture (ISA)**. The ISA is founded on a **RISC philosophy** to ensure hardware simplicity, a clean pipeline, and high clock speeds at low power.

Key architectural features of the ISA are:

- **32-bit Architecture:** All registers and instructions are 32 bits.
- **32 General-Purpose Registers:** A x0-x31 register file, with x0 hardwired to zero.
- **Fixed-Length Instructions:** All instructions are 32 bits, simplifying fetch and decode stages.
- **Load/Store Architecture:** All operations occur on registers; memory is accessed only via explicit *GETW* (load) and *PUTW* (store) instructions.
- **Domain-Specific Extensions:** The base integer ISA is powerfully augmented with a custom instruction set specifically to accelerate our target workloads. This includes:
    - **DSP/Vector Instructions:** A high-throughput *VMA* (Vector Multiply-Accumulate), *VSIMD_ADD* and *VHUSUM*.

- **Math Instructions:** Dedicated floating-point FROOT (Square Root) and *FSIN* (Sine) for audio synthesis and normalization.

1.3 Supported Instruction Classes

This microarchitecture is designed to execute all instruction classes defined in our ISA:

- **R-type:** Register-to-Register integer operations (e.g., SUM rd, rs1, rs2).
- **I-type:** Immediate-to-Register integer operations (e.g., SUMI rd, rs1, imm12) and memory loads (e.g., GETW rd, imm12(rs1)).
- **S-type:** Memory store operations (e.g., PUTW rs2, imm12(rs1)).
- **B-type:** Conditional branch operations (e.g., BRANCHEQ rs1, rs2, imm12).
- **J-type:** Unconditional jump operations (e.g., JUMPL rd, imm20).
- **Custom R4-type:** Our specialized VMAC rd, rs1, rs2 instruction, which implicitly reads rd as a third source operand, enabling a three-operand operation.
- **Custom F-type:** Floating-point operations (e.g., FROOT rd, rs1) that utilize a dedicated FPU.

1.4 Design Objective

The primary design objective is not to build the fastest possible general-purpose CPU. Instead, our objective is to achieve **maximum power efficiency** for our specific target workloads.

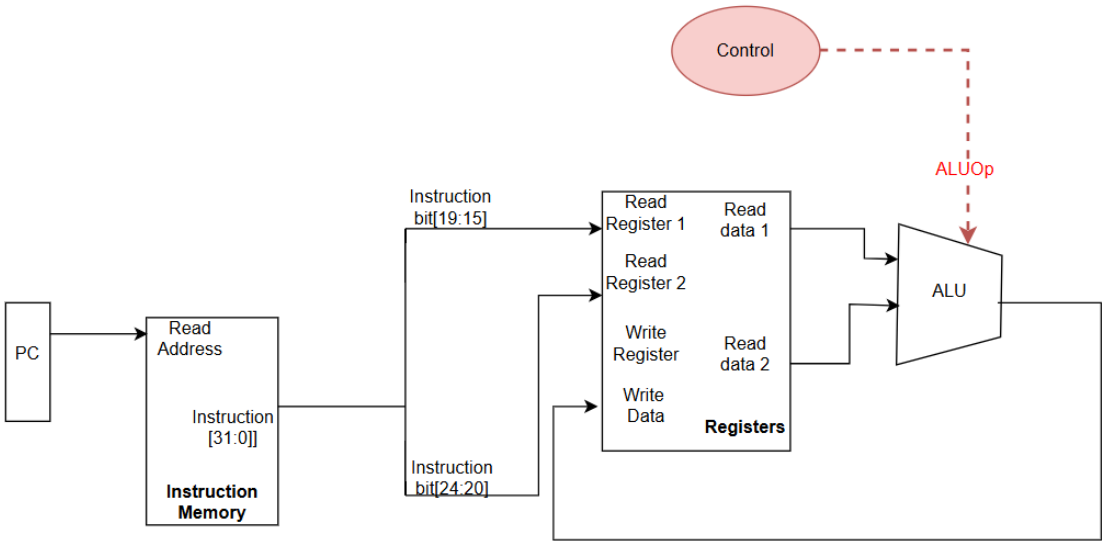
For a low-cost mobile device, **power is the dominant cost**. A power-hungry processor requires a larger battery, more complex power management circuitry, and potentially a heat sink, all of which dramatically increase the final cost of the phone.

Our strategy is to **minimize the CPI (Cycles Per Instruction)** for the critical DSP/AI loops by implementing them in custom hardware. This allows our processor to meet real-time performance targets (e.g., processing audio without lag) at a **significantly lower clock frequency**. A lower clock frequency is the single most effective way to reduce power consumption. Therefore, by trading a small amount of specialized silicon (our custom instructions) for a massive reduction in required clock speed, we directly achieve our "low-cost" objective.

2. Incremental Datapath Design

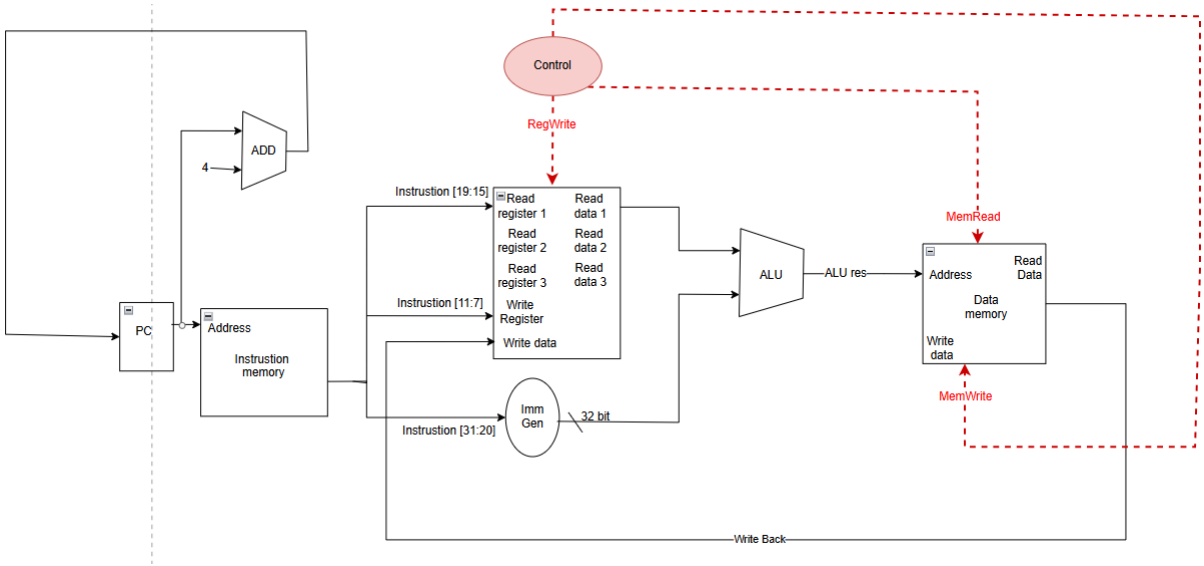
This section describes the datapath incrementally, showing the hardware components and connections required for each major instruction class.

2.1 R-type (e.g., SUM rd, rs1, rs2):



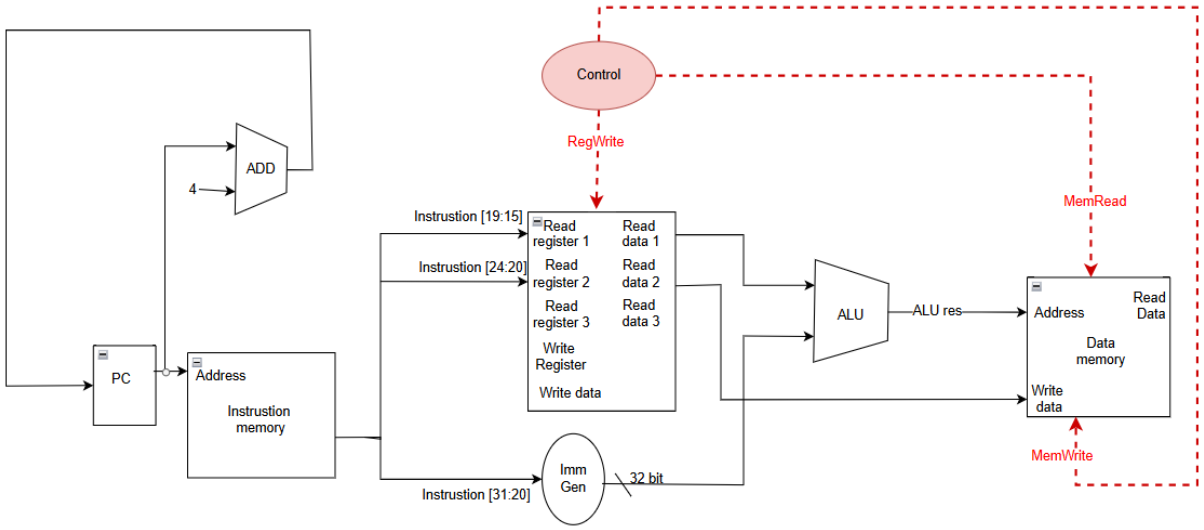
Execution Explanation: An R-type instruction (Register-to-Register) is fetched by the PC from Instruction Memory. The instruction is decoded, and the rs1 and rs2 addresses are sent to the Register File. The two 32-bit values are read from the file and passed as inputs to the Main ALU. The ALU performs the operation specified by the ALUOp control signal (e.g., "ADD" for SUM). The computed 32-bit result is then routed back to the Register File's WriteData port and saved into the destination register specified by the rd field.

2.2 I-type Load (e.g., GETW rd, imm12(rs1)):



Execution Explanation: A load instruction calculates a memory address. It fetches the base address from rs1 in the Register File and simultaneously sign-extends the 12-bit immediate offset. The Main ALU adds these two values to compute the final 32-bit effective address. This address is sent to Data Memory. The MemRead control signal is asserted, and the memory unit fetches the 32-bit word at that location. This data is then sent to the Register File and written into the rd register.

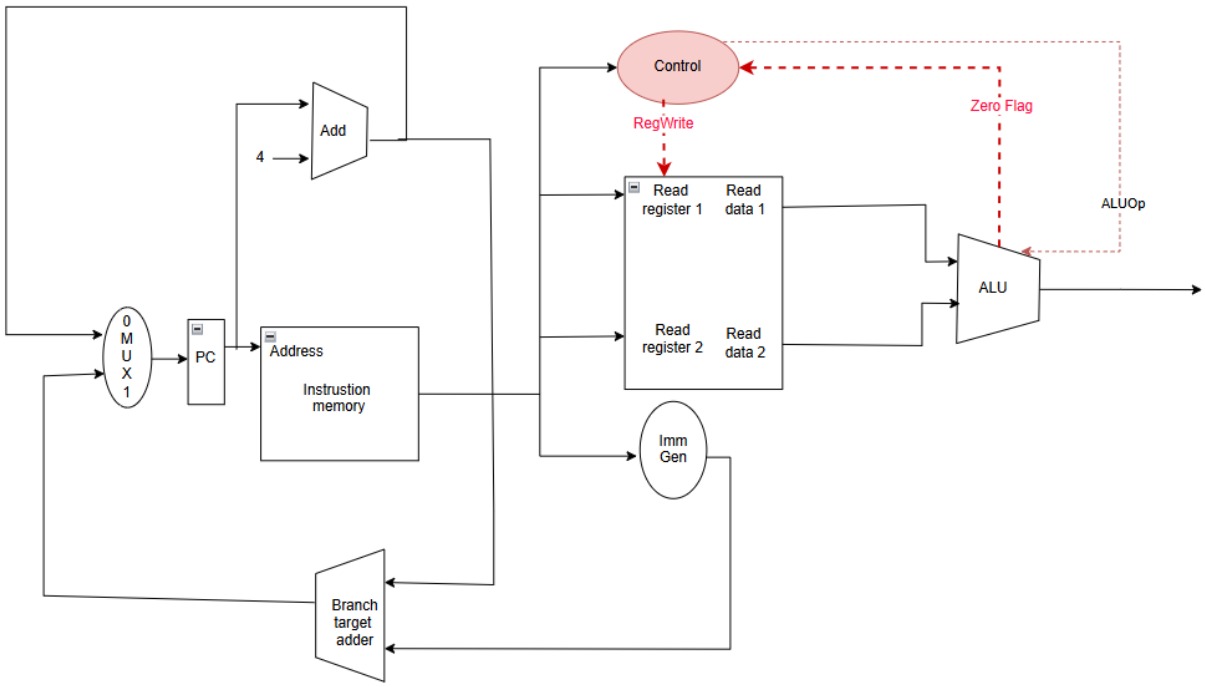
2.3 S-type Store (e.g., PUTW rs2, imm12(rs1)):



Execution Explanation: A store instruction calculates its target address identically to a load. However, it also reads a second register, rs2, to get the data that needs to be saved. The

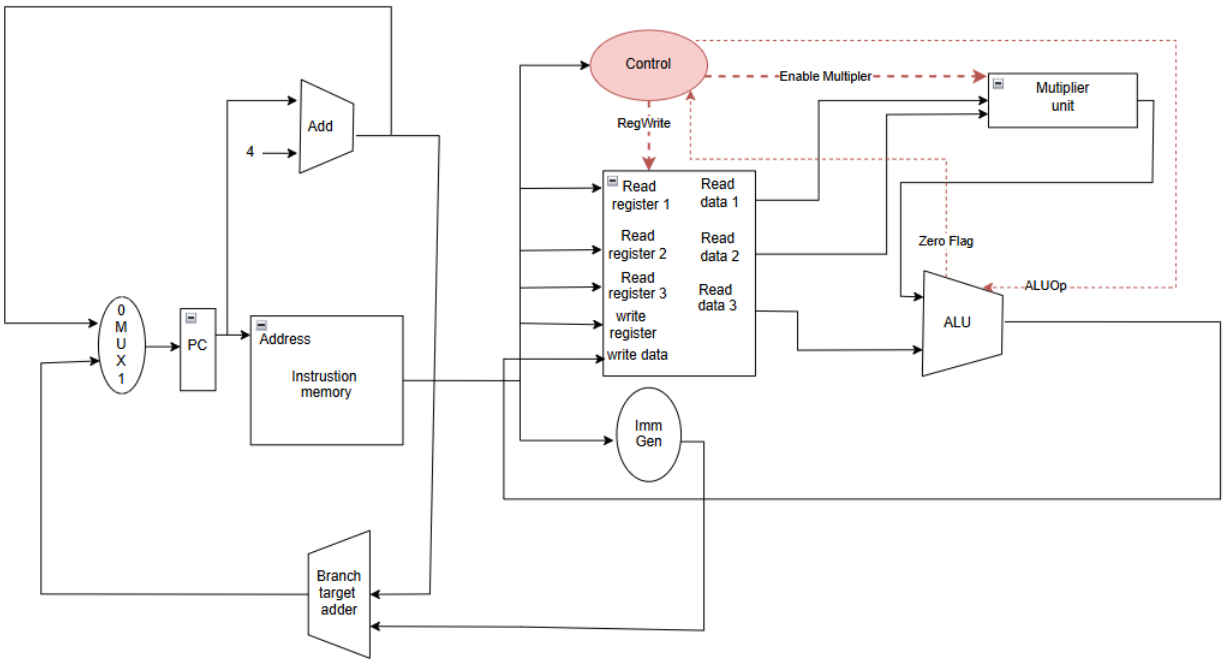
ALU_Result is sent to the Data Memory's address port, while the value from rs2 is sent to its WriteData port. The MemWrite control signal is asserted, and the Data Memory unit writes the data from rs2 into the specified memory location. No register is modified.

2.4 B-type Branch (e.g., BRANCHEQ rs1, rs2, imm12):



Explanation: A branch instruction must make a decision. It reads rs1 and rs2 and sends them to the ALU, which performs a subtraction. If the Zero flag is set (meaning rs1 == rs2), the branch is taken. In parallel, the branch target address is computed by adding the sign-extended immediate to the PC. A control signal, conditioned on the Zero flag, selects the next PC address: either PC+4 (if no branch) or the BranchTargetAddress (if branch is taken).

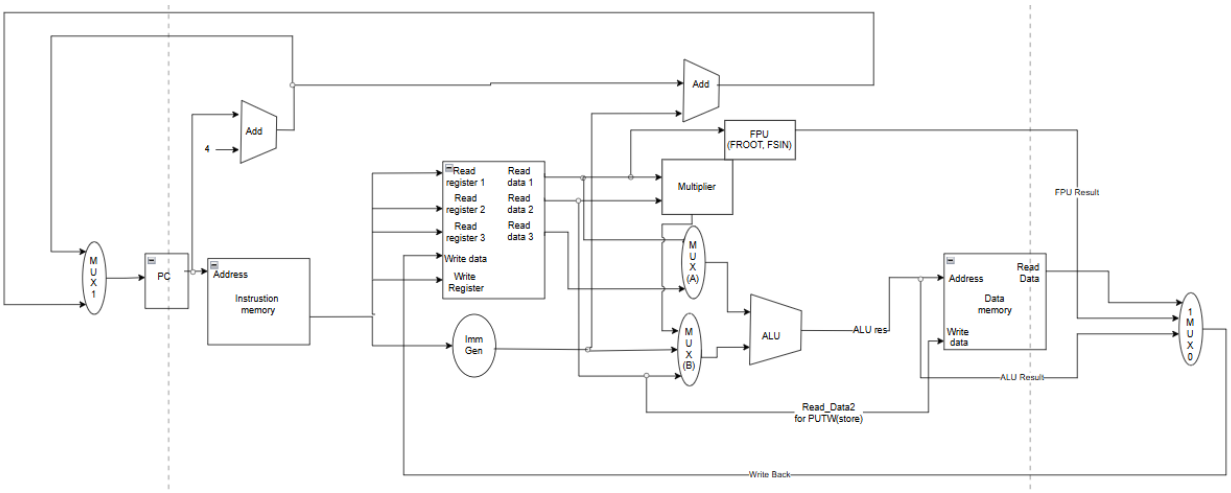
2.5 Custom R4-type (VMAC rd, rs1, rs2):



Execution Explanation: Our custom VMAC instruction is the core of our AI acceleration. It uses a specialized **3-port Register File** to read rs1, rs2, and rd *simultaneously*. The rs1 and rs2 values are sent to a dedicated Multiplier Unit to compute their product. In parallel, the current (old) value of the rd register is sent to Input A of the Main ALU. The result from the Multiplier is then fed as the *second* operand to the Main ALU, which is instructed to "ADD". The final result, R[rd] + (R[rs1] * R[rs2]), is written back to the rd register. This completes a full multiply-accumulate operation in a single pass, which would normally take 3 or 4 separate instructions.

3. Unified Single-Cycle Datapath

A single-cycle datapath is capable of executing all instructions. Now by combine the components from Section 2 and adding **Multiplexers (MUXes)** to select the correct data source for each component based on the instruction being executed. We present the unified Single-Cycle Datapath as shown below:

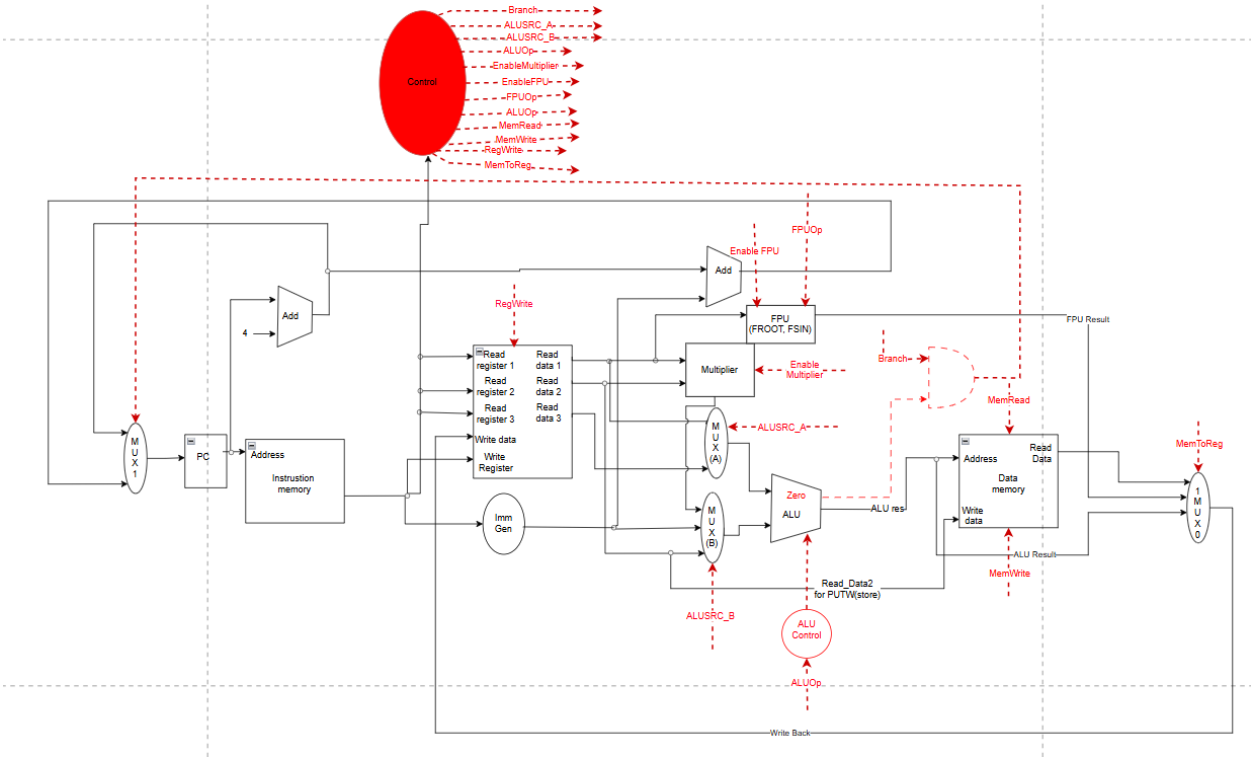


Execution Flow

In this single-cycle design, every instruction (even SUM) takes as long as the slowest possible instruction (e.g., GETW or FROOT). The instruction is fetched, decoded, and all control signals are set at once. The data flows through all MUXes and functional units, and the final result is written back to a register or memory at the end of one long clock cycle.

4. Control Unit Design

The Control Unit is a combinational logic block responsible for managing the entire datapath. It is implemented as a "red ellipse" on the diagram, connected "by name" to all components.



Control Signal Truth Table

This table defines the state of the primary control signals for our key instruction classes. This is the "logic" inside the Control Unit.

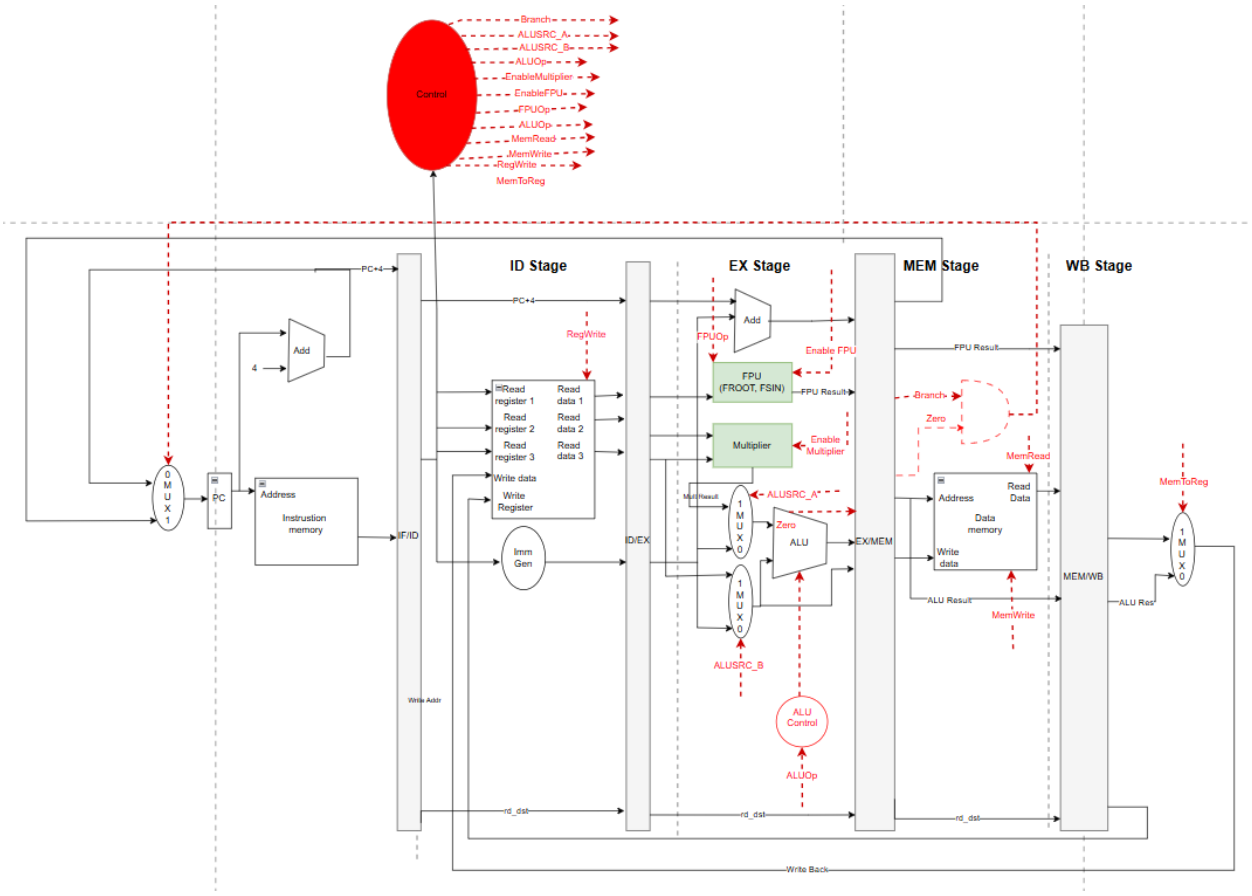
Instruction	ALUSrc_A	ALUSrc_B	ALUOp	EnableMulti	Enable FPU	MemRead	MemWrite	RegWrite	MemToReg	Branch
Description	1-bit MUX. 0=rs1, 1=rd	2-bit MUX. 0=rs2, 1=Imm, 2=Mult	4-bit. e.g., ADD, SUB	1-bit On/Off	1-bit On/Off	1-bit On/Off	1-bit On/Off	1-bit On/Off	2-bit MUX. 0=ALU, 1=Mem, 2=FPU	1-bit On/Off
SUM	0	0	ADD	Off	Off	Off	Off	**On**	0	Off
SUMI	0	1	ADD	Off	Off	Off	Off	**On**	0	Off
GETW	0	1	ADD	Off	Off	**On**	Off	**On**	**1**	Off
PUTW	0	1	ADD	Off	Off	Off	**On**	**Off**	X	Off
BRANCHEQ	0	0	SUB	Off	Off	Off	Off	**Off**	X	**On**
VMAC	**1**	**2**	ADD	**On**	Off	Off	Off	**On**	0	Off
FROOT	X	X	X	Off	**On**	Off	Off	**On**	**2**	Off

(Note: 'X' = Don't Care, as the signal's value doesn't matter for this instruction)

5. Pipelined Implementation

The single-cycle datapath is simple but slow. We convert it into a 5-stage pipeline (IF, ID, EX, MEM, WB) to increase instruction throughput from 1/CPI to 1.

5.1 Updated Datapath Diagram (Pipelined):



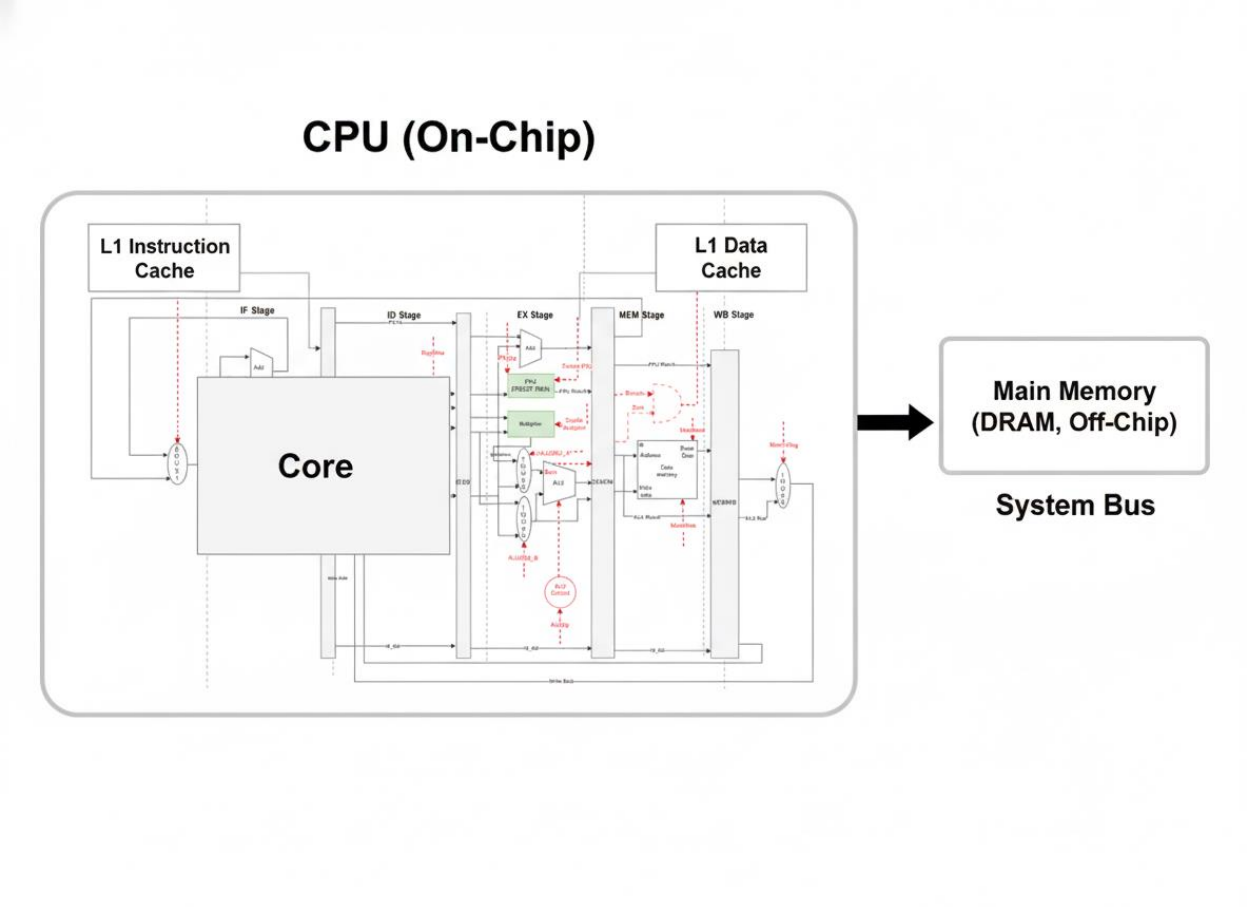
5.2 Hazard Discussion and Optimizations

Pipelining creates conflicts, or "hazards," that must be resolved.

- **Data Hazards:**
  - **Problem:** An instruction needs a result from a *previous* instruction that has not yet reached the Write-Back stage (e.g., *SUM x3, x1, x2* followed by *SUM x4, x3, x5*).
  - **Solution (Forwarding/Bypassing):** We implement a **Forwarding Unit**. This is a hardware block that compares the *rs* addresses of the instruction in the *EX* stage with the rd addresses of the instructions in the MEM and WB stages. If there is a match, it controls MUXes before the ALU inputs to send the result "forward" in time, directly from the *EX/MEM or MEM/WB* registers, bypassing the *Register File.*
  - **Solution (Stalling): Check:** A **Load-Use Hazard** (e.g., GETW x3, 0(x1) followed by SUM x4, x3, x5) cannot be fully resolved by forwarding, as the data is not available from memory until the end of the MEM stage. In this case, our Hazard Detection Unit (part of Control) will **stall** the pipeline for one cycle by "freezing" the PC and IF/ID register and inserting a "bubble" (a NOP) into the ID/EX register.
- **Control Hazards:**
  - **Problem:** A branch instruction (BRANCHEQ) does not determine the next PC address until the EX stage (after the ALU comparison). By then, two more instructions have already been fetched.
  - **Solution:** We use a simple **static "predict-not-taken"** strategy. The pipeline continues fetching from PC+4. If the branch *is* taken, the Control Unit will **"flush"** the two incorrectly fetched instructions from the IF/ID and ID/EX registers (by clearing their control signals) and update the PC to the correct branch target address.
- **Structural Hazards (Multi-Cycle):**
  - **Problem:** Our custom FPU instructions (FROOT, FSIN) are complex and cannot finish in one EX cycle.
  - **Solution:** The FPU is a **multi-cycle, pipelined unit**. When it begins execution, it asserts a Stall signal *to* the Control Unit. The Control Unit then **freezes the IF and ID stages**, preventing new instructions from being fetched or decoded. This inserts "bubbles" into the EX stage, allowing the FPU to complete its operation without a new FPU instruction arriving and causing a conflict. Once the FPU is ready, it de-
  - asserts the Stall signal, and the pipeline resumes.

6. Memory Hierarchy Design

The memory hierarchy is not an optional add-on; it is a **mission-critical component** for achieving our low-power performance goals. Our specialized VMAC and FPU units are "hungry" and would be useless if they were constantly stalled waiting for data.



Description and Rationale: Our processor *requires* a **Harvard-style Level 1 (L1) cache** architecture, meaning we have two separate caches:

1. **L1 Instruction Cache (e.g., 16KB):** This cache provides instructions to the IF stage. Our AI workloads are dominated by small, tight loops (like the dot_product_workload). The L1-I cache ensures these loops are fetched from main memory *once* and then execute entirely from the fast, low-power on-chip cache, maximizing fetch bandwidth.

2. **L1 Data Cache (e.g., 16KB):** This cache services all GETW and PUTW requests from the MEM stage. Our workloads are "data-hungry," processing large, contiguous streams of audio samples or image pixels. Without an L1-D cache, *every single* GETW or VMAC operation would stall the pipeline for hundreds of cycles to fetch from slow DRAM. This would completely nullify the gains from our custom instructions. The L1-D cache ensures that data is prefetched and available at high speed, keeping the VMAC and FPU units "fed" and operating at maximum efficiency

7. Conclusion

This microarchitecture design directly achieves our primary objective of a low-cost, power-efficient processor for the Lesotho market. The design's philosophy is to trade a small, one-time silicon cost for massive, continuous gains in power efficiency.

The key architectural trade-off—a more complex Execute stage and the inclusion of a 3-port Register File—is strongly justified. This complexity enables our custom VMAC instruction to execute in a single pass. This single optimization halves the instruction counts and more than halves the clock cycles required for our most critical AI loops, allowing the entire processor to run at a significantly lower clock frequency. This, in turn, is the dominant factor in reducing power consumption and, therefore, the total system cost of the mobile device.

The resulting design is not a general-purpose processor; it is a highly specialized, domain-specific engine. It is optimized from the ground up to solve real-world accessibility problems using AI, providing computational power that would otherwise be out of reach for a low-cost device.

8. References

- Patterson, D. A., & Hennessy, J. L. (2017). *Computer Organization and Design RISC-V Edition*.
- A. Waterman and K. Asanović, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA Specification*, Version 20191213, RISC-V Foundation, 2024. [Online]. Available: https://riscv.org/technical/specifications/
- National University of Lesotho. (2025). *Processor Design and Implementation* [Lecture slides]. Department of Mathematics and Computer Science.CS3520 Computer Organization and Architecture I.